

DS520 - DEEP LEARNING

IMAGE CAPTIONING USING CNN AND LSTM



Introduction

Image captioning is a cutting-edge application of deep learning that combines computer vision and natural language processing to generate descriptive captions for images. It enables machines to interpret visual content and articulate it in human-readable language. This technology has a wide range of applications, including assistive tools for the visually impaired, automatic tagging for social media platforms, and content moderation. Image captioning plays a vital role in enhancing accessibility and improving user experiences in various domains.

Paper Overview

PAPER 1: Image_caption_using_CNN and LSTM

Uses pre-trained CNN model, VGG16, and Xception model in combination with LSTM combination to predict captions. The paper has used a Greedy Algorithm to predict captions which we have also opted for in our project (predicting only one possible word). The paper also compared the performance of both models, demonstrating that Xception outperforms VGG16 in accuracy. However, due to computational constraints, we opted to use VGG16 for our implementation.

PAPER 2: IMAGE CAPTION GENERATION USING DEEP LEARNING TECHNIQUE

This paper uses the InceptionV3 model as a pre-trained CNN for feature extraction, combined with LSTM to predict image captions. The paper implements the Beam Search algorithm for caption generation, which is more efficient than the Greedy Algorithm as it considers multiple possible sequences at each step, resulting in better-quality captions. But it is more memory-intensive. So we opted to use the Greedy Algorithm in our project

PAPER 3: Show and Tell: A Neural Image Caption Generator

The paper uses the VGG16 model in combination with LSTM, similar to the previous approach. it also incorporates an attention module, which enhances the captioning process by creating context-aware captions.

Our Approach

We have used a combination of **VGG16** Convolutional Neural Network (**CNN**) for feature extraction and Bidirectional Long Short Term Memory (**Bi - LSTM**) network with **Attention module** for sequence generation.

Methodology

1. Image Feature Extraction:

Pretrained **VGG16** is used as the CNN model, where the last fully connected layer (classification layer) is removed to extract high-dimensional feature vectors of size (1, 4096) for each image. These features will be used as input to our model.

2. Caption Preprocessing:

Captions are tokenized, cleaned, and padded to ensure uniform input length. Special tokens **startseq** and **endseq** are added to define the start and end of each caption sequence. This is done to ensure the model knows when to start and end predicting words.

3. Training Phase:

A bi-directional LSTM network is used in combination with attention mechanisms, to process both image features and the previously generated words to predict the next word in the caption sequence.

4. Prediction Phase:

The model **generates captions word by word**, iteratively using the predicted word as input for the next step until the **endseq** or the maximum caption length is reached.

Libraries

- **TensorFlow/Keras:** For building and training CNN and LSTM models.
- **Pandas and NumPy:** For data handling and preprocessing.
- **Matplotlib and PIL:** For visualization and image processing.

Algorithms

- **VGG16 (CNN):** Pretrained CNN Model. Extracts high-level image features.
- **LSTM with Attention:** Processes sequential data and focuses on relevant image regions or prior captions during prediction.

Dataset

The **flickr8k** dataset from Kaggle is used for this project. The dataset has an images folder and a caption file.

- **Image Folder:** 8,091 images
- **Captions File:** Five captions per image (40455 total captions)

```
image,caption
1000268201_693b08cb0e.jpg,A child in a pink dress is climbing up a set of stairs in an entry way .
1000268201_693b08cb0e.jpg,A girl going into a wooden building .
1000268201_693b08cb0e.jpg,A little girl climbing into a wooden playhouse .
1000268201_693b08cb0e.jpg,A little girl climbing the stairs to her playhouse .
1000268201_693b08cb0e.jpg,A little girl in a pink dress going into a wooden cabin .
```

Captions file snippet

Implementation Details

Phase - 1: Load and Restructure VGG16 Model

VGG16 Model was loaded and the last classification layer was removed because we don't want to classify objects. Instead, we want to use the extracted features from images and use those features as input to the prediction model(CNN + LSTM)

```
model = VGG16()  
  
# Restructuring the model to remove the last classification layer, this will give us access to the output features of the model  
model = Model(inputs=model.inputs, outputs=model.layers[-2].output)
```

Phase - 2: Image Features Extraction

Images from the dataset are fed to the restructured VGG16 model which in return extracts the features in (1,4096) vector for each image. These features are stored in the **image_features** dictionary with image_id as key and features as value.

```
Image ID: 1000268201_693b08cb0e, Feature Shape: (1, 4096)  
Image ID: 1001773457_577c3a7d70, Feature Shape: (1, 4096)  
Image ID: 1002674143_1b742ab4b8, Feature Shape: (1, 4096)  
Image ID: 1003163366_44323f5815, Feature Shape: (1, 4096)  
Image ID: 1007129816_e794419615, Feature Shape: (1, 4096)  
Image ID: 1007320043_627395c3d8, Feature Shape: (1, 4096)  
Image ID: 1009434119_febe49276a, Feature Shape: (1, 4096)  
Image ID: 1012212859_01547e3f17, Feature Shape: (1, 4096)  
Image ID: 1015118661_980735411b, Feature Shape: (1, 4096)  
Image ID: 1015584366_dfcec3c85a, Feature Shape: (1, 4096)
```

Phase 3: Preprocessed and Tokenized Caption Data

In this phase, a dictionary was created to save all captions as values corresponding to their respective image as keys and then we preprocessed captions:

- Converted to lowercase
- Removed extra spaces

-
- Added a token '**startseq**' at the start of each caption and '**endseq**' at the end of each caption so that when we train and predict captions model knows when to start and when to stop making the prediction

Tokenized Captions: The tokenizer processes all captions by creating a list of unique words and assigning each word a corresponding integer value. Additionally, the tokenizer keeps track of two key attributes:

- **Maximum Caption Length (33):** Represents the length of the longest caption, which is used to pad shorter captions to ensure a consistent input size of (None, max_caption_length) for the model.
- **Vocabulary Size (8768):** Refers to the total number of unique words in the caption dataset, used to set the dimensionality of the output layer, as the model generates a probability distribution over the entire vocabulary.

This tokenization process is crucial for model prediction. During prediction, the tokenizer converts the input caption sequence (e.g., "caption so far") into integers. After the model predicts the next word's index, the tokenizer reverses this process by converting the index back into the corresponding word. This ensures the model can handle sequences effectively and generate meaningful captions.

```
# after preprocess of text
image_to_captions_mapping['1026685415_0431cbf574']

➡ ['startseq black dog carries green toy in his mouth as he walks through the grass endseq',
   'startseq black dog carrying something through the grass endseq',
   'startseq black dog has blue toy in its mouth endseq',
   'startseq dog in grass with blue item in his mouth endseq',
   'startseq wet black dog is carrying green toy through the grass endseq']
```

Phase 4: Model Design

- **Image Input (CNN):** Features extracted from VGG16 in phase 2 are passed through a dense layer and projected into sequences using a repeat vector and passed to bi-directional LSTM for deeper understanding of spatial relationships between features
- **Caption Input (LSTM):** Captions(Sequence) generated so far are passed as input followed by Embedding layers which process tokenized captions, followed by bi-directional LSTMs for sequence learning.
- **Attention Mechanism:** Attention is applied to focus on relevant parts of the input.
- **Output Layer:** A dense layer *predicts the next word as a probability distribution* over the vocabulary.

Phase 5: Model Training

The dataset is split into **90%** training and **10%** testing.

The model is trained with categorical **cross-entropy loss** using a batch size of 32 for 50 epochs. Data generators are used to feed batches of image features, current captions, and target words for training. Data generators feed data input to the model in batches.

Phase 6: Caption Prediction

- Captions are iteratively generated word-by-word.
- The process stops when ‘endseq’ is predicted or the maximum caption length is reached.
- The model uses the context from both the **image features** and the **current caption sequence** to predict the next word.

Caption Prediction Workflow

Step 1: Initialize the Caption

The caption generation process begins by initializing the sequence with the special token **startseq**. This acts as the starting point for the model.

Step 2: Prepare Input for the Model

- Convert the current caption sequence (e.g., caption so far) into its integer form using the tokenizer.
- Pad the sequence with zeros to make it match the fixed maximum caption length (e.g., 34 words).

Step 3: Predict the Next Word

- The processed caption sequence (so far) and the pre-extracted image features are fed into the trained model.
- The model predicts a probability distribution ($\hat{y}$) over all vocabulary words for the next word.

Step 4: Select the Predicted Word

- The word index with the highest probability is selected as the next word in the caption.
- This index is converted back into a word using the tokenizer's `get_word_from_index` function.

Step 5: Update the Caption

The predicted word is added to the current caption sequence.

Step 6: Check for Termination

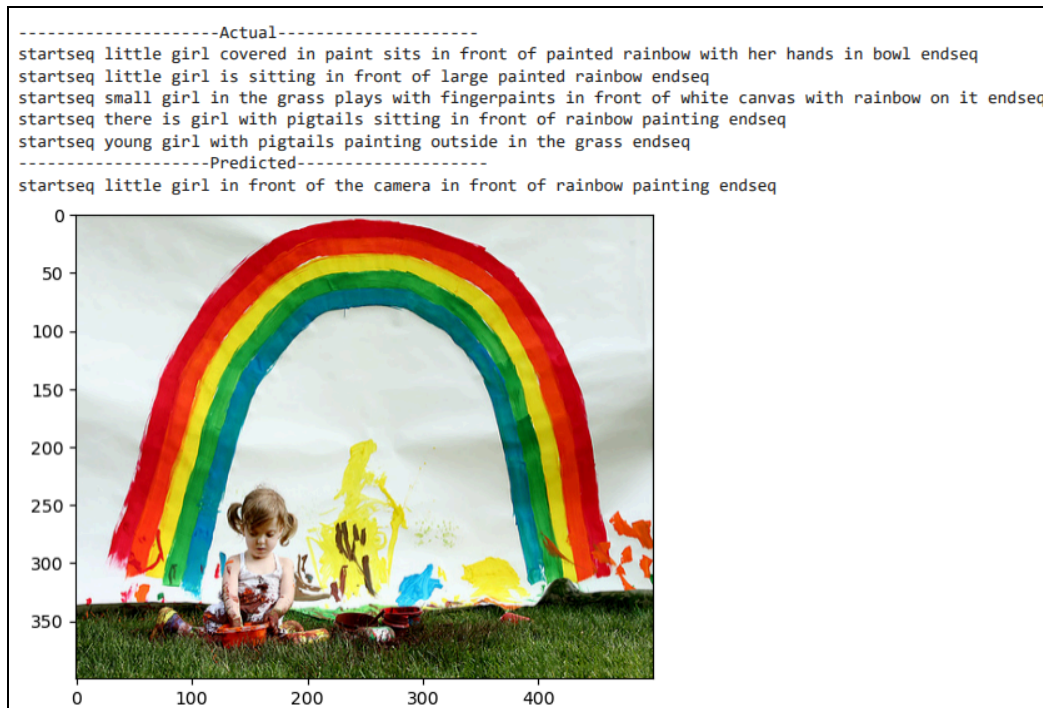
- If the predicted word is **endseq** or the **maximum caption length** is reached, the loop terminates.

-
- Otherwise, the updated caption sequence is used as input for the next iteration.

Step 7: Return the Generated Caption

- The final caption, now containing the sequence of words from startseq to endseq, is returned as the output.

Output



References

Paper 1:

[Image Caption using CNN & LSTM | Request PDF](#) (*This paper needs access. pdf copy is submitted on Canvas)

Paper 2:

https://www.irjmets.com/uploadedfiles/paper/issue_6_june_2022/26209/final/fin_irjmets1655531403.pdf

Paper 3:

[\[1411.4555\] Show and Tell: A Neural Image Caption Generator](#)